

Les conteneurs vus du cloud

ECR, ECS et Fargate : exécuter un conteneur sans gérer de serveur

CL-AWS2 — Jour 4 (jour 31 du cursus)

Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- Expliquer avec vos mots ce qu'est une **image** de conteneur et un **conteneur** en exécution — sans encore savoir en fabriquer (ce sera le rôle du bloc CL-CONT).
- Pousser et tirer une image dans **ECR**, le registre d'images managé d'AWS.
- Décrire les quatre objets d'**ECS** : cluster, task definition, task, service.
- Déployer un conteneur public sur **Fargate**, derrière un ALB, et l'arrêter proprement.
- Choisir entre **Lambda**, **Fargate** et **EC2** pour une charge de travail donnée.
- Situer **App Runner** dans l'offre AWS.

Fil conducteur : encore un cran de plus vers le « managé » — après la VM (EC2) et la fonction (Lambda), le **processus empaqueté**.

Plan de la journée

1. **C'est quoi, un conteneur ?** — vulgarisation, juste ce qu'il faut pour aujourd'hui.
2. **ECR** — le registre d'images : pousser, tirer, authentifier.
3. **ECS et Fargate** — cluster, task definition, service ; déployer derrière un ALB.
4. **Choisir son compute** — Lambda vs Fargate vs EC2, et App Runner en survol.

Au programme : 1 démo (Fargate + ALB), 2 exercices, quiz de fin de journée.

1. C'est quoi, un conteneur ?

8 slides de vulgarisation — le vrai bloc Docker arrive au bloc 13

Avertissement honnête avant de commencer

Aujourd'hui, nous regardons les conteneurs **du point de vue d'AWS** : comment le cloud les héberge, les facture, les expose.

- Vous n'écrirez **pas de Dockerfile** aujourd'hui.
- Vous ne construirez **pas d'image** aujourd'hui.
- Vous allez **consommer** des images déjà fabriquées (publiques), comme on consomme une AMI sans savoir la construire.

Le bloc **13 — CL-CONT (Docker + Kubernetes, 7 jours)** couvrira la fabrication d'images, docker compose, puis Kubernetes et EKS.

Objectif du jour : quand quelqu'un dira « on déploie ça en conteneur sur Fargate », vous saurez exactement de quoi il parle, ce que ça coûte et comment l'administrer.

Le problème de départ : « ça marche sur ma machine »

Souvenez-vous de vos déploiements de StockLine :

- **TP1** : installer Python, le venv, les dépendances, l'unité systemd, nginx... à la main, en suivant une procédure.
- **TP2** : la même chose, automatisée dans un script `user-data` — mais toujours dépendante de l'OS de l'AMI (version d'Ubuntu, d'apt, de Python...).

À chaque fois, le risque est le même : **l'application dépend de la machine qui l'héberge**. Une version de Python qui change, un paquet manquant, et rien ne démarre.

L'idée du conteneur : **empaqueter l'application AVEC tout son environnement** (Python, dépendances, fichiers, configuration) dans un seul artefact, qui s'exécute à l'identique partout.

VM et conteneur : ce qui est embarqué

MACHINE VIRTUELLE	CONTENEUR
+-----+	+-----+
Application (StockLine)	Application (StockLine)
+-----+	+-----+
Python, dépendances	Python, dépendances
+-----+	+-----+
OS COMPLET (Ubuntu,	(rien d'autre : le noyau
noyau, systemd, ssh...)	est PARTAGÉ avec l'hôte)
+-----+	+-----+
Hyperviseur	Moteur de conteneurs
+-----+	+-----+
Matériel	OS hôte + matériel
+-----+	+-----+
Démarrage : minutes	Démarrage : secondes
Taille : Go	Taille : dizaines/centaines de Mo

L'image : un paquet figé

Une **image** de conteneur, c'est un **fichier d'archive** (en réalité un empilement de couches) qui contient :

- l'application (le code de StockLine, par exemple) ;
- son runtime (Python 3.12, uvicorn) ;
- ses dépendances (fastapi, sqlalchemy...) ;

- sa configuration de démarrage (« lance `uvicorn app.main:app` »).

Analogie avec ce que vous connaissez déjà :

Vous connaissez	Équivalent conteneur
<code>stockline.zip</code> + venv + procédure d'installation	l' image (tout-en-un, figé)
AMI (modèle de VM)	image = « AMI de processus », beaucoup plus légère
<code>stockline.zip</code> v2, v3... sur S3	les tags d'image : <code>stockline:1.0</code> , <code>stockline:1.1</code>

Une image est **immuable** : on ne la modifie pas, on en construit une nouvelle version.

Le conteneur : un processus isolé

Quand on **exécute** une image, on obtient un **conteneur** : un simple **processus Linux**, mais isolé du reste de la machine.

- Il voit **son propre système de fichiers** (celui de l'image), pas celui de l'hôte.
- Il a **son propre réseau** (sa propre adresse IP dans les cas qui nous occupent).
- Il ne voit **pas les autres processus** de la machine.
- Il partage le **noyau** de l'hôte — c'est pour ça qu'il démarre en secondes.

Rapport image / conteneur — à retenir absolument :

```

image      →  exécution  →  conteneur
(le paquet,      (le processus vivant,
sur un registre)      sur une machine)

1 image peut donner N conteneurs (comme 1 AMI → N instances)
```

Le registre : la bibliothèque d'images

Les images vivent dans un **registre** (registry) : un service de stockage versionné, spécialisé pour les images.

- **Docker Hub** : le registre public historique (`nginx`, `python`, `postgres`...).
- **Amazon ECR Public** (`public.ecr.aws`) : le registre public d'AWS.
- **Amazon ECR** (privé) : votre registre à vous, dans votre compte — on le voit dans la section 2.

Deux verbes à connaître :

- **push** : **pousser** une image vers un registre (comme `aws s3 cp` vers un bucket) ;
- **pull** : **tirer** une image depuis un registre (fait automatiquement par le service qui exécute le conteneur).

Une image est identifiée par son **URI complète** : `123456789012.dkr.ecr.eu-west-3.amazonaws.com/abc-stockline:1.0` (registre / dépôt / tag)

Vocabulaire minimal du jour

Terme	Définition en une ligne
Image	Paquet figé : application + runtime + dépendances + commande de démarrage
Tag	Étiquette de version d'une image (:1.0, :latest)
Conteneur	Processus isolé issu de l'exécution d'une image
Registre	Service qui stocke et distribue les images (Docker Hub, ECR)
Repository (dépôt)	Dans un registre : l'emplacement d'UNE application et de ses tags
Pull / Push	Tirer une image depuis / pousser une image vers un registre
Orchestrateur	Service qui décide OÙ et COMBIEN de conteneurs tournent (ECS, Kubernetes)

C'est tout ce qu'il vous faut pour la journée. Le reste (Dockerfile, couches, volumes, réseaux Docker) : bloc 13.

Pourquoi le cloud adore les conteneurs

Du point de vue d'un administrateur cloud, le conteneur apporte :

- **Densité** : plusieurs applications isolées sur une même machine → moins d'instances, moins de coûts.
- **Portabilité** : la même image tourne sur votre poste, sur EC2, sur Fargate, sur Kubernetes, chez un autre fournisseur.
- **Déploiements rapides et réversibles** : déployer = tirer une nouvelle image ; revenir en arrière = re-tirer l'ancienne.
- **Standard ouvert** (OCI) : pas de format propriétaire, tout l'écosystème parle le même langage.

Et la question qui nous occupe aujourd'hui : **qui fait tourner ces conteneurs, et qui administre les machines en dessous ?** C'est exactement l'offre ECS / Fargate.

2. ECR — le registre d'images d'AWS

ECR : Elastic Container Registry

ECR est le registre d'images managé d'AWS. Pensez-y comme à « **un S3 spécialisé pour les images de conteneurs** » :

- **Privé par défaut** : vos images ne sont visibles que dans votre compte (politiques IAM).
- **Régional** : un dépôt vit dans une région (pour nous : eu-west-3).
- **Intégré** : ECS, EKS, Lambda (images), App Runner tirent depuis ECR nativement.
- **Fonctions managées** : scan de vulnérabilités des images, règles de cycle de vie (supprimer les vieux tags), réplication inter-régions.

Facturation : **0,10 \$/Go/mois** de stockage (tarif indicatif) + transfert sortant. Ordre de grandeur : négligeable en formation.

L'unité de rangement est le **repository** : un dépôt par application (**abc-stockline**), plusieurs tags dedans (**1.0, 1.1, latest**).

Créer un dépôt et s'authentifier

Créer un dépôt (une ligne) :

```
aws ecr create-repository \  
  --repository-name "$PREFIX-nginx-demo" \  
  --region eu-west-3
```

Le registre exige une **authentification** avant tout push/pull manuel. On demande à AWS un jeton (valable 12 h) et on le passe au client de conteneurs :

```
aws ecr get-login-password --region eu-west-3 \  
| docker login --username AWS \  
  --password-stdin 123456789012.dkr.ecr.eu-west-3.amazonaws.com
```

Pas de mot de passe en dur : le jeton vient de vos identifiants IAM. Les services AWS (ECS, Fargate), eux, s'authentifient **tout seuls via leur rôle IAM** — vous n'aurez jamais à faire ce login pour eux.

Le cycle re-taguer / pousser (fait par le formateur en démo)

Pour alimenter ECR **sans construire d'image**, on prend une image publique et on la re-tague vers notre dépôt :

```
# 1. Tirer une image publique (nginx, serveur web)  
docker pull public.ecr.aws/nginx/nginx:1.27  
  
# 2. La re-taguer avec l'URI de NOTRE dépôt ECR  
docker tag public.ecr.aws/nginx/nginx:1.27 \  
  123456789012.dkr.ecr.eu-west-3.amazonaws.com/abc-nginx-demo:1.27  
  
# 3. La pousser  
docker push 123456789012.dkr.ecr.eu-west-3.amazonaws.com/abc-nginx-demo:1.27  
  
# 4. Vérifier  
aws ecr list-images --repository-name abc-nginx-demo --region eu-west-3
```

docker n'est ici qu'un **outil de transport** (comme **aws s3 cp**). La fabrication d'images viendra au bloc 13. Vos machines n'ont pas Docker : cette étape est une **démo formateur** ; votre TP utilisera directement l'image publique.



Démo 4-1 (partie 1) — ECR en action

Le formateur montre, vous observez et posez des questions.

Au programme :

1. Création d'un dépôt ECR `fmt-nginx-demo` dans `eu-west-3`.
2. Authentification `aws ecr get-login-password | docker login`.
3. Pull de l'image publique `public.ecr.aws/nginx/nginx:1.27`, re-tag, push.
4. Visite guidée de la console ECR : tags, taille, résultat du scan de vulnérabilités.

Fichier : `demos/08-cl-aws2/demo-4-1-fargate.md` (actes 1 et 2)

Question à garder en tête pendant la démo : *à quoi ressemblerait ce cycle pour livrer StockLine v1.1 en production ?*

3. ECS et Fargate

L'orchestrateur de conteneurs « made in AWS »

ECS : Elastic Container Service

Vous avez une image. Il faut maintenant répondre à des questions d'**exploitation** :

- Sur quelle machine exécuter le conteneur ? Avec combien de CPU/RAM ?
- Qui le **redémarre** s'il meurt à 3 h du matin ?
- Comment en exécuter **3 exemplaires** derrière un répartiteur de charge ?
- Comment déployer la version suivante **sans coupure** ?

Répondre à ces questions, c'est le travail d'un **orchestrateur**. AWS en propose deux :

- **ECS** : l'orchestrateur propriétaire d'AWS — simple, profondément intégré, notre sujet du jour ;
- **EKS** : Kubernetes managé — le standard du marché, plus riche et plus complexe (bloc 13/TP5).

ECS est la porte d'entrée idéale : les concepts sont les mêmes, la marche est moins haute.

Les 4 objets d'ECS

```

CLUSTER (le périmètre logique : "où")
+-----+
|
|  SERVICE (le contrat : "combien, en continu")
|  desired count = 2, attaché à l'ALB
|  +-----+ +-----+
|  | TASK #1 | | TASK #2 |
|  | (conteneur nginx | | (conteneur nginx |
|  | en exécution) | | en exécution) |
|  |
|
|
|

```



Les 4 objets, définis

Objet	Rôle	Analogie déjà vue
Cluster	Regroupement logique où s'exécutent les tâches	le VPC des conteneurs (organisationnel)
Task definition	La recette , versionnée : quelle(s) image(s), CPU, RAM, ports, variables d'env, logs	le Launch Template de l'ASG (TP2)
Task	Une exécution de la recette : un ou plusieurs conteneurs vivants	l'instance EC2 lancée depuis le template
Service	Le contrat de disponibilité : « maintiens N tâches, remplace celles qui meurent, branche-les sur l'ALB »	l'Auto Scaling Group

À retenir : on ne lance presque jamais une task « à la main » en production — on déclare un **service**, et ECS s'occupe du reste. Exactement la philosophie de l'ASG au TP2.

Anatomie d'une task definition

Extrait commenté de `code/08-cl-aws2/task-definition.json` :

```
{
  "family": "abc-nginx-demo",           // nom de la recette (versionnée : :1, :2...)
  "requiresCompatibilities": ["FARGATE"], // où elle a le droit de tourner
  "networkMode": "awsvpc",              // chaque tâche reçoit sa propre ENI/IP
  "cpu": "256",                          // 256 = 0,25 vCPU (unités ECS)
  "memory": "512",                       // 512 Mio de RAM
  "containerDefinitions": [{
    "name": "nginx",
    "image": "public.ecr.aws/nginx/nginx:1.27", // QUELLE image tirer
    "portMappings": [{ "containerPort": 80 }], // port exposé par le processus
    "logConfiguration": { // stdout → CloudWatch Logs
      "logDriver": "awslogs",
      "options": { "awslogs-group": "/ecs/abc-nginx-demo", "...": "..." }
    }
  }]
}
```

Chaque enregistrement crée une **révision** (`abc-nginx-demo:1`, `:2...`) : les recettes sont immuables, comme les images.

Le service : votre « systemd + ASG » des conteneurs

Le **service** ECS maintient l'état désiré, en continu :

- **desiredCount**: 2 → ECS garantit 2 tâches vivantes. Une tâche meurt ? Il en relance une.
- Attaché à un **target group** de l'ALB → chaque tâche est enregistrée/désenregistrée automatiquement, health checks compris.
- Déploiement **rolling** par défaut : nouvelle révision de task definition → ECS démarre les nouvelles tâches, vérifie leur santé, puis arrête les anciennes.
- Peut s'**auto-scaler** (nombre de tâches) sur une métrique CloudWatch, comme l'ASG du TP2.

```
aws ecs create-service \  
  --cluster "$PREFIX-demo" \  
  --service-name "$PREFIX-nginx-svc" \  
  --task-definition "$PREFIX-nginx-demo:1" \  
  --desired-count 2 \  
  --launch-type FARGATE ...
```

Fargate vs EC2 : qui fournit les machines ?

Une tâche ECS doit s'exécuter **quelque part**. Deux « launch types » :

	EC2 launch type	Fargate
Machines	VOS instances EC2 dans le cluster	aucune visible : AWS fournit la capacité
Vous administrez	AMI, patching, scaling du parc	rien (que la tâche)
Facturation	les instances (même vides !)	à la tâche, à la seconde
Densité/coût	meilleur si parc bien rempli	plus cher à l'unité, zéro gâchis
Cas d'usage	gros volumes stables, besoins GPU	équipes petites, charges variables

Fargate = le serverless des conteneurs : vous déclarez CPU/RAM par tâche, AWS place le conteneur, vous payez la durée d'exécution.

Dans ce cursus (et dans la plupart des PME) : **Fargate d'abord**, EC2 launch type seulement si le calcul de coûts le justifie.

Réseau : le mode awsvpc

En mode **awsvpc** (obligatoire avec Fargate), **chaque tâche reçoit sa propre interface réseau (ENI)** dans un de vos sous-réseaux :

- la tâche a sa **propre IP privée** dans le VPC ;
- on lui attache un **security group**, exactement comme à une instance EC2 ;
- l'ALB lui envoie le trafic en mode **target type ip** (et non **instance** comme au TP2).

Conséquences pratiques :

- vos réflexes VPC/SG du bloc AWS1 s'appliquent **tels quels** : SG de l'ALB ouvert en 80 au monde, SG des tâches ouvert en 80 **depuis le SG de l'ALB uniquement** ;
- une tâche dans un sous-réseau **public** a besoin de **assignPublicIp=ENABLED** pour tirer son image depuis Internet (ou d'une NAT GW / d'endpoints VPC si sous-réseau privé — plus propre en production, plus cher en formation).

Logs et supervision d'une tâche

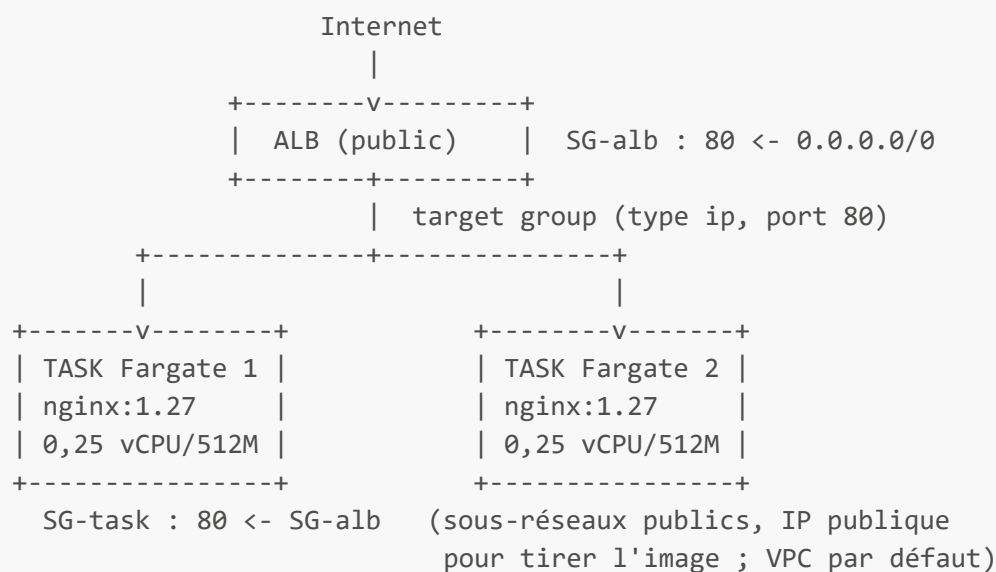
Un conteneur bien élevé écrit ses journaux sur sa **sortie standard** (stdout/stderr) — pas dans des fichiers.

- Le driver **awslogs** de la task definition envoie tout vers **CloudWatch Logs** (groupe `/ecs/<famille>`).
- Métriques CPU/mémoire par service dans **CloudWatch** (namespace `AWS/ECS`) — alarmes possibles, comme au J8 d'AWS1.
- `aws ecs describe-tasks` + l'onglet « Logs » de la console = vos premiers réflexes de dépannage.

```
# Suivre les logs d'une tâche en quasi temps réel
aws logs tail "/ecs/$PREFIX-nginx-demo" --follow --region eu-west-3
```

Réflexe d'admin : **une tâche qui boucle en démarrage/arrêt** = allez lire `stoppedReason` dans `describe-tasks`, puis les logs. 90 % des cas : image introuvable, port faux, ou mémoire insuffisante.

Architecture cible de la démo





Démo 4-1 (partie 2) — nginx sur Fargate derrière un ALB

Déroulé (le formateur pilote, tout en CLI, `eu-west-3`) :

1. Création du cluster, enregistrement de la task definition (`task-definition.json`).
2. Création SG, ALB, target group (type `ip`), listener.
3. `create-service --desired-count 2` → observation du démarrage des tâches.
4. Test dans le navigateur via le DNS de l'ALB ; arrêt sauvage d'une tâche → **ECS la remplace tout seul**.
5. Teardown complet avec `teardown-fargate.sh`.

Fichiers : `demos/08-cl-aws2/demo-4-1-fargate.md`, `code/08-cl-aws2/deploy-fargate.sh`, `teardown-fargate.sh`

Vous rejouerez ce déploiement vous-mêmes en fin de démo, avec votre `$PREFIX`.



Fargate se paie à la durée

Fargate facture chaque tâche **à la seconde**, sur deux compteurs (tarifs indicatifs eu-west-3, 2025 — vérifiez la calculatrice AWS) :

- **~0,047 \$ par vCPU et par heure**
- **~0,0051 \$ par Go de RAM et par heure**

Notre tâche de démo (0,25 vCPU / 0,5 Go), qui tourne 24 h/24 :

```
CPU   : 0,25 × 0,047 = 0,01175 $/h
RAM   : 0,5 × 0,0051 = 0,00255 $/h
Total ≈ 0,0143 $/h → ≈ 10,4 $/mois PAR TÂCHE (≈ 21 $ pour nos 2 tâches)
+ 1'ALB : ≈ 0,027 $/h ≈ 19 $/mois + LCU
```

Une tâche Fargate oubliée coûte, même sans trafic. Un service avec `desired-count 2` relancera éternellement ce que vous tuez à la main : pour arrêter, on passe le service à 0 **puis** on supprime. Teardown systématique en fin de journée.

Arrêter proprement : la checklist

Dans l'ordre (sinon ECS ressuscite ce que vous supprimez) :

```
# 1. Plus aucune tâche désirée
aws ecs update-service --cluster "$PREFIX-demo" \
  --service "$PREFIX-nginx-svc" --desired-count 0

# 2. Supprimer le service (--force accepte les tâches en cours d'arrêt)
aws ecs delete-service --cluster "$PREFIX-demo" \
  --service "$PREFIX-nginx-svc" --force

# 3. Supprimer ALB, listener, target group (payés à l'heure aussi !)
```

```
# 4. Supprimer le cluster
aws ecs delete-cluster --cluster "$PREFIX-demo"

# 5. Vérifier qu'il ne reste RIEN qui tourne
aws ecs list-tasks --cluster "$PREFIX-demo" --region eu-west-3
```

Le script `teardown-fargate.sh` fait tout cela — mais vous devez **comprendre l'ordre** pour le jour où le script n'existera pas.



Exercice 4-1 — Lambda, Fargate ou EC2 ?

En binômes, 30 minutes.

Six charges de travail réelles vous sont décrites (API, batch, site, traitement long, tâche planifiée, legacy). Pour chacune :

1. choisissez **Lambda, Fargate ou EC2** ;
2. justifiez en une ou deux phrases (durée, trafic, contraintes, coût) ;
3. notez le critère qui a **fait basculer** votre décision.

Restitution : chaque binôme défend UN de ses choix devant le groupe.

Fichier : `exercices/08-cl-aws2/exercice-4-1-choisir-son-compute.md`

Faites l'exercice **avant** de voir le tableau de décision qui suit — c'est volontaire.

4. Choisir son compute

Lambda vs Fargate vs EC2 — et App Runner

Le tableau de décision

Critère	Lambda	Fargate	EC2
Unité	fonction (événement)	tâche (conteneur)	instance (VM)
Durée max d'exécution	15 min	illimitée	illimitée
Démarrage	ms → s (cold start)	~30-60 s	minutes
Facturation	à la requête + ms	à la seconde de tâche	à l' heure d'instance
Trafic idéal	épisodique, en pics	continu ou soutenu	continu, stable, massif
Vous administrez	le code	le conteneur	OS + tout le reste
Contrainte forte	15 min, taille paquet	image requise	patching, capacité
StockLine en...	API à faible trafic (TP3)	API à trafic soutenu	3-tiers du TP2

Règle simple : **événementiel et court** → **Lambda** ; **continu et conteneurisé** → **Fargate** ; **besoin de la machine (OS, GPU, licences, très gros volume stable)** → **EC2**.

Le même StockLine, trois façons de l'héberger

Une API comme StockLine peut vivre dans les trois mondes — le **contexte** décide :

- **EC2 + ASG (TP2)** : trafic soutenu et prévisible, besoin de maîtriser l'OS (agents, tuning PostgreSQL local). Coût fixe, administration maximale.
- **Lambda + API GW (TP3, à venir)** : trafic en pics ou faible (quelques requêtes/min). Coût quasi nul au repos, mais cold starts et 15 min max.
- **Fargate + ALB (TP5, à venir, via EKS)** : trafic continu, équipe qui veut des conteneurs (portabilité, mêmes images du poste à la prod), sans gérer de VM.

Il n'y a **pas de bonne réponse absolue** — il y a un bon choix **par charge de travail**. C'est exactement ce qu'évalue l'examen Cloud Practitioner... et votre futur employeur.

App Runner, en survol

AWS App Runner : encore un cran au-dessus de Fargate dans le « managé ».

- Vous donnez : une **image** (ECR) ou un **dépôt de code** source.
- App Runner s'occupe de : build éventuel, déploiement, **HTTPS**, nom de domaine, répartition de charge, **auto-scaling**, et même la mise en veille à trafic nul.
- Vous ne voyez **ni cluster, ni task definition, ni ALB**.

	ECS/Fargate	App Runner
Contrôle (VPC, ALB, réglages fins)	complet	limité
Mise en route	~1 h la première fois	~10 minutes
Cas d'usage	plateformes d'équipe, prod outillée	petite API/web à mettre en ligne vite

À connaître pour le nommer et le situer — nous ne l'utiliserons pas dans le cursus (ECS/EKS couvrent nos besoins et le marché de l'emploi).



Exercice 4-2 — Lire une task definition et calculer son coût

Individuel, 30 minutes.

À partir d'une task definition fournie (une API Python en 0,5 vCPU / 1 Go) :

1. répondez à 6 questions de lecture (image, port, logs, que se passe-t-il si...) ;
2. calculez le **coût mensuel** du service en 2 tâches 24/7, puis en 2 tâches aux heures ouvrées (11 h/j, 5 j/7) ;
3. comparez à une alternative EC2 t3.small et concluez.

Fichier : [exercices/08-cl-aws2/exercice-4-2-task-definition-et-cout.md](#)

Calculette et tarifs indicatifs fournis dans l'énoncé. On pose les calculs — vous en aurez besoin au J6 pour la facture Negoce+.

Récap de la journée

- **Image** = **paquet figé** (app + runtime + dépendances), **conteneur** = **processus isolé** qui exécute cette image. Fabrication des images : bloc 13.
- **ECR** : registre privé managé — dépôts, tags, authentification par jeton IAM.
- **ECS** : cluster (où), task definition (quoi), task (exécution), **service** (contrat de disponibilité — votre ASG des conteneurs).
- **Fargate** : AWS fournit les machines, vous payez **la tâche à la seconde** — et une tâche oubliée coûte. Teardown : service à 0 → delete → ALB → cluster.
- Choix du compute : **événementiel court** → **Lambda** ; **continu conteneurisé** → **Fargate** ; **besoin de la machine** → **EC2**. App Runner = raccourci tout-managé.

Le trio « automatiser, superviser, sécuriser » vaut aussi pour les conteneurs : SG sur les tâches, logs dans CloudWatch, scripts de déploiement/teardown.

Quiz de fin de journée

10 questions — répondez sur papier, correction ensemble

Quiz — questions 1 et 2

Question 1. Quelle est la différence entre une image et un conteneur ?

A. Aucune, ce sont deux noms pour la même chose B. L'image est le paquet figé, le conteneur est le processus en exécution issu de l'image C. Le conteneur est le paquet, l'image est le processus D. L'image ne sert que sur Docker Hub, le conteneur que sur AWS

Question 2. Dans ECS, quel objet décrit « quelle image exécuter, avec combien de CPU et de mémoire » ?

A. Le cluster B. Le service C. La task definition D. Le target group

Quiz — questions 3 et 4

Question 3. Quel objet ECS garantit que 2 tâches restent en vie et les remplace en cas de panne ?

A. Le service B. La task definition C. Le repository ECR D. Le listener de l'ALB

Question 4. Avec le launch type Fargate, qui administre les serveurs qui exécutent les tâches ?

A. Vous, via un Auto Scaling Group d'instances EC2 B. AWS — aucune instance n'est visible dans votre compte C. Le fournisseur de l'image (nginx, par exemple) D. Personne, les tâches tournent dans le navigateur

Quiz — questions 5 et 6

Question 5. Comment Fargate est-il facturé ?

A. Au nombre d'images stockées B. Un forfait mensuel par cluster C. À la seconde, selon le vCPU et la mémoire alloués à chaque tâche D. À la requête HTTP reçue

Question 6. En mode réseau `awsvpc`, qu'est-ce qu'une tâche reçoit ?

A. Une adresse IP publique obligatoire B. Sa propre interface réseau (ENI) avec IP privée et security group C. L'IP de l'instance EC2 hôte D. Un nom DNS Route 53 automatique

Quiz — questions 7 et 8

Question 7. Vous voulez arrêter définitivement un service Fargate qui a 2 tâches. Quelle est la première étape correcte ?

A. Tuer les 2 tâches avec `stop-task` et attendre B. Supprimer le cluster directement C. Passer le `desired-count` du service à 0 D. Supprimer l'image dans ECR

Question 8. Un traitement de données dure 45 minutes, une fois par nuit. Quel service est le PLUS adapté ?

A. Lambda, car c'est le moins cher B. Une tâche Fargate planifiée, car Lambda est limité à 15 minutes C. Une instance EC2 allumée 24 h/24 D. App Runner

Quiz — questions 9 et 10

Question 9. À quoi sert `aws ecr get-login-password` ?

A. À créer un utilisateur IAM pour Docker B. À obtenir un jeton temporaire pour s'authentifier auprès du registre ECR C. À chiffrer les images poussées dans ECR D. À réinitialiser le mot de passe du compte AWS

Question 10. Quelle affirmation sur App Runner est vraie ?

A. Il nécessite de créer soi-même un ALB et un cluster ECS B. Il déploie une image ou un dépôt de code avec HTTPS et auto-scaling gérés pour vous C. C'est le nom du launch type EC2 d'ECS D. Il ne fonctionne qu'avec des fonctions Lambda

Réponses expliquées demain matin — et dans le guide formateur.

À demain !

Jour 5 : l'infrastructure en code

Aujourd'hui vous avez déployé un service Fargate **en une dizaine de commandes**. Demain, on fait mieux : **toute une infrastructure décrite dans un seul fichier YAML**, déployée, mise à jour et détruite en une commande — CloudFormation, et vos premiers pas vers Terraform.

Ce soir : vérifiez votre teardown (`aws ecs list-clusters`, `aws elbv2 describe-load-balancers`) — **zéro tâche, zéro ALB**.